

# Linked List, Tree, Graph & Grid

Struktur data yang sering muncul sebagai bentuk dunia: rantai, hierarki, jaringan, dan peta.

Tujuan dokumen ini: bukan menghafal tanda-tanda soal, tapi memahami gerak berpikir pattern. Setiap pattern dijelaskan dengan contoh dunia nyata, contoh teknologi, konsep komputasi, alasan kenapa contoh itu sama dengan pattern, dan template Dart minimal.

# Cara Membaca Dokumen Ini

Jangan baca seperti kamus keyword. Baca seperti latihan membangun intuisi. Pattern LeetCode itu mirip alat kerja: tiap alat punya cara bergerak, situasi natural, dan bentuk data yang cocok.

Untuk tiap pattern, pakai urutan baca ini: makna manusiawi -> contoh dunia nyata -> contoh teknologi -> konsep komputasi -> bentuk LeetCode -> template Dart -> pertanyaan cek pemahaman.

| Pattern         | Makna inti                                    |
|-----------------|-----------------------------------------------|
| Linked List     | rantai node yang dihubungkan pointer          |
| Tree DFS/BFS    | hierarki yang dijelajahi dalam atau per level |
| BST             | tree dengan aturan kiri kecil kanan besar     |
| Graph Traversal | jaringan node dan relasi                      |
| Matrix/Grid     | peta 2D sebagai graph terselubung             |

# Linked List - rantai node yang dihubungkan pointer

## Makna manusiawinya

Linked list adalah rangkaian node. Setiap node tahu node berikutnya. Tidak seperti array, akses random tidak murah; kekuatannya ada pada manipulasi pointer.

## Contoh nyata non-teknologi

### Rantai gerbong kereta

Setiap gerbong terhubung ke gerbong berikutnya. Untuk membalik urutan, sambungan antar gerbong harus dibalik satu per satu.

### Antrian manual pakai kertas

Setiap orang menunjuk siapa setelahnya. Kalau satu orang dihapus, orang sebelumnya harus menunjuk ke orang setelah yang dihapus.

## Contoh nyata di dunia teknologi

### LRU Cache

Sering memakai doubly linked list untuk memindahkan item terbaru ke depan dan membuang item paling lama.

### Playlist media

Next/previous track bisa direpresentasikan sebagai linked structure.

## Konsep komputasinya

Secara komputasi, linked list menuntut kehati-hatian dengan prev, curr, next. Banyak soal memakai dummy node, fast-slow pointer, reverse pointer.

## Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena urutan dibentuk oleh sambungan, bukan index. Mengubah urutan berarti mengubah link.

## Bentuknya di LeetCode

- Reverse Linked List
- Linked List Cycle
- Merge Two Sorted Lists
- Remove Nth From End
- Reorder List

## Template Dart minimal

```
class ListNode {
  int val;
  ListNode? next;
  ListNode(this.val, [this.next]);
}

ListNode? reverseList(ListNode? head) {
  ListNode? prev;
  var curr = head;
```

```
while (curr != null) {  
    final next = curr.next;  
    curr.next = prev;  
    prev = curr;  
    curr = next;  
}  
return prev;  
}
```

## Kalau masih bingung, tanya begini

- Apa pointer yang harus dipertahankan sebelum link diubah?
- Apakah dummy node bisa menyederhanakan edge case?
- Apakah perlu fast-slow pointer?

## Mini drill pemahaman

1. Kenapa reverse list butuh next sementara?
2. Apa edge case head null?
3. Kenapa remove head lebih mudah dengan dummy?

# Tree DFS/BFS - hierarki yang dijelajahi dalam atau per level

## Makna manusiawinya

Tree adalah graph tanpa siklus dengan arah hierarki: parent-child. DFS cocok untuk informasi dari anak ke parent. BFS cocok untuk level/order per kedalaman.

## Contoh nyata non-teknologi

### Struktur organisasi

CEO -> VP -> Manager -> Staff. Kalau mau tahu tinggi organisasi, pakai DFS. Kalau mau lihat semua orang per level jabatan, pakai BFS.

### Daftar isi buku

Bab, subbab, sub-subbab membentuk tree. Lu bisa menyelam ke satu bab atau baca semua judul level pertama dulu.

## Contoh nyata di dunia teknologi

### DOM/UI widget tree

Frontend dan Flutter punya tree: widget punya child. Render dan layout sering berjalan lewat traversal tree.

### File system

Folder dan file adalah tree. Search bisa DFS atau BFS tergantung kebutuhan.

## Konsep komputasinya

Secara komputasi, Tree DFS sering memakai recursion dan mengembalikan informasi. Tree BFS memakai queue dan memproses node per level.

## Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena struktur hierarkinya jelas. Ada akar, cabang, dan daun. Cara traversal bergantung pada pertanyaan: kedalaman atau level.

## Bentuknya di LeetCode

- Max Depth, Diameter, Path Sum
- Level Order Traversal
- Invert Binary Tree
- Balanced Tree
- Lowest Common Ancestor

## Template Dart minimal

```
class TreeNode {
  int val;
  TreeNode? left;
  TreeNode? right;
  TreeNode(this.val, [this.left, this.right]);
}

int maxDepth(TreeNode? root) {
```

```

    if (root == null) return 0;
    final left = maxDepth(root.left);
    final right = maxDepth(root.right);
    return 1 + (left > right ? left : right);
}

import 'dart:collection';

List<List<int>> levelOrder(TreeNode? root) {
  if (root == null) return [];
  final result = <List<int>>[];
  final queue = Queue<TreeNode>()..add(root);

  while (queue.isNotEmpty) {
    final size = queue.length;
    final level = <int>[];
    for (int i = 0; i < size; i++) {
      final node = queue.removeFirst();
      level.add(node.val);
      if (node.left != null) queue.add(node.left!);
      if (node.right != null) queue.add(node.right!);
    }
    result.add(level);
  }
  return result;
}

```

## Kalau masih bingung, tanya begini

- Apakah jawaban parent butuh informasi child?
- Apakah hasilnya per level?
- Traversal mana yang membuat state paling sederhana?

## Mini drill pemahaman

1. Untuk max depth, kenapa DFS natural?
2. Untuk level order, kenapa BFS natural?
3. Apa bedanya preorder dan postorder?

# BST - tree dengan aturan kiri kecil kanan besar

## Makna manusiawinya

Binary Search Tree adalah tree yang punya aturan urutan: semua di kiri lebih kecil, semua di kanan lebih besar. Ini membuat tree bisa dipakai seperti binary search berbentuk hierarki.

## Contoh nyata non-teknologi

### Rak buku yang diatur alfabetis bercabang

Kalau judul lebih kecil, cari ke kiri; kalau lebih besar, cari ke kanan. Lu tidak perlu cek semua rak.

### Decision tree angka

Setiap node adalah pertanyaan: lebih kecil atau lebih besar?

## Contoh nyata di dunia teknologi

### Indexing sederhana

Konsep tree terurut dipakai dalam banyak struktur indexing, meski database modern lebih sering memakai variasi seperti B-tree.

### Kth smallest

Karena inorder BST menghasilkan urutan sorted, kita bisa mencari elemen ke-kecil.

### Konsep komputasinya

Secara komputasi, BST memanfaatkan invariant  $low < node < high$ . Inorder traversal menghasilkan nilai sorted.

### Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena aturan urutan mengarahkan pencarian. Tanpa aturan ini, BST cuma binary tree biasa.

## Bentuknya di LeetCode

- Validate BST
- Search in BST
- Kth Smallest
- LCA of BST
- Insert into BST

## Template Dart minimal

```
bool isValidBST(TreeNode? root, [int? low, int? high]) {  
  if (root == null) return true;  
  if (low != null && root.val <= low) return false;  
  if (high != null && root.val >= high) return false;  
  return isValidBST(root.left, low, root.val) &&  
    isValidBST(root.right, root.val, high);  
}
```

## Kalau masih bingung, tanya begini

- Apakah property BST berlaku global, bukan cuma parent-child?

- Apakah inorder membantu?
- Apakah bisa pakai batas low/high?

### **Mini drill pemahaman**

1. Kenapa node kiri harus lebih kecil dari semua ancestor kanan?
2. Apa hasil inorder BST?
3. Bedakan BST dan binary tree biasa.



# Graph Traversal - jaringan node dan relasi

## Makna manusiawinya

Graph merepresentasikan entitas dan hubungan. Tidak harus hierarki. Bisa punya cycle, banyak path, disconnected component, dan arah relasi.

## Contoh nyata non-teknologi

### Peta pertemanan

Orang adalah node, pertemanan adalah edge. Ada circle, mutual friend, dan komunitas.

### Jaringan jalan

Kota adalah node, jalan adalah edge. Bisa ada banyak rute menuju tujuan.

## Contoh nyata di dunia teknologi

### Dependency service

Service A memanggil B dan C. Relasi antar service membentuk graph.

### Package dependency

Library punya dependency ke library lain. Cycle bisa jadi masalah.

## Konsep komputasinya

Secara komputasi, graph sering direpresentasikan sebagai adjacency list. Traversal butuh visited agar tidak muter tanpa akhir.

## Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena fokusnya relasi antar entitas. Tree adalah kasus khusus graph, tapi graph lebih umum dan lebih liar.

## Bentuknya di LeetCode

- Clone Graph
- Number of Connected Components
- Course Schedule
- Pacific Atlantic Water Flow
- Graph Valid Tree

## Template Dart minimal

```
Map<int, List<int>> buildGraph(List<List<int>> edges) {  
  final graph = <int, List<int>>{};  
  for (final e in edges) {  
    final a = e[0], b = e[1];  
    graph.putIfAbsent(a, () => []).add(b);  
    graph.putIfAbsent(b, () => []).add(a);  
  }  
  return graph;  
}
```

## Kalau masih bingung, tanya begini

- Node dan edge-nya apa?

- Apakah graph directed atau undirected?
- Apakah bisa ada cycle?
- Apakah butuh visited?

### **Mini drill pemahaman**

1. Modelkan Instagram follower sebagai graph directed.
2. Kenapa visited wajib?
3. Apa beda connected component dan shortest path?

# Matrix/Grid - peta 2D sebagai graph terselubung

## Makna manusiawinya

Grid adalah graph yang disamakan sebagai tabel 2D. Setiap cell bisa dianggap node, dan tetangganya adalah atas, bawah, kiri, kanan, atau diagonal.

## Contoh nyata non-teknologi

### Peta kota kotak-kotak

Setiap blok adalah cell. Dari satu blok lu bisa jalan ke blok sebelahnya.

### Area banjir

Air menyebar dari cell ke tetangga. Ini natural sebagai BFS/DFS grid.

## Contoh nyata di dunia teknologi

### Image processing

Pixel adalah cell. Operasi flood fill menyebar ke pixel tetangga.

### Game map

Karakter bergerak dari tile ke tile. Pathfinding di tile map adalah graph traversal.

## Konsep komputasinya

Secara komputasi, grid traversal memakai boundary check, visited/markings, dan directions. Kesalahan umum: out of bounds dan lupa menandai visited.

## Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena cell punya hubungan lokal ke tetangga. Walau input berupa matrix, pikirkan sebagai graph.

## Bentuknya di LeetCode

- Number of Islands
- Max Area of Island
- Rotting Oranges
- Surrounded Regions
- Word Search
- Shortest Path in Binary Matrix

## Template Dart minimal

```
int numIslands(List<List<String>> grid) {  
  final rows = grid.length, cols = grid[0].length;  
  final dirs = [[1,0], [-1,0], [0,1], [0,-1]];  
  
  void dfs(int r, int c) {  
    if (r < 0 || c < 0 || r >= rows || c >= cols) return;  
    if (grid[r][c] != '1') return;  
    grid[r][c] = '0';  
    for (final d in dirs) dfs(r + d[0], c + d[1]);  
  }  
}
```

```

int count = 0;
for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        if (grid[r][c] == '1') {
            count++;
            dfs(r, c);
        }
    }
}
return count;
}

```

## Kalau masih bingung, tanya begini

- Apa cell valid?
- Arah gerak apa saja?
- Apakah perlu BFS untuk jarak atau DFS untuk komponen?
- Bagaimana menandai visited?

## Mini drill pemahaman

1. Kenapa island sama dengan connected component?
2. Kapan BFS lebih cocok daripada DFS di grid?
3. Apa edge case grid kosong?

## Penutup

Mastery bukan berarti menghafal semua tanda soal. Mastery berarti saat melihat masalah, lu bisa memodelkan gerak masalahnya: apakah ia menyebar, menyelam, menjaga window, mencari boundary, mengingat key, menggabungkan kelompok, atau menyimpan subproblem.

Cara latihan: ambil satu pattern, pahami analoginya, kerjakan 3-5 soal mudah, tulis ulang konsepnya dengan bahasa sendiri, lalu campur dengan pattern lain supaya otak belajar mengenali bentuk masalah tanpa spoiler.